TP OOP

Name : CHHEAN RAVY
ID : e20241154
Group : I3_AMS_1A

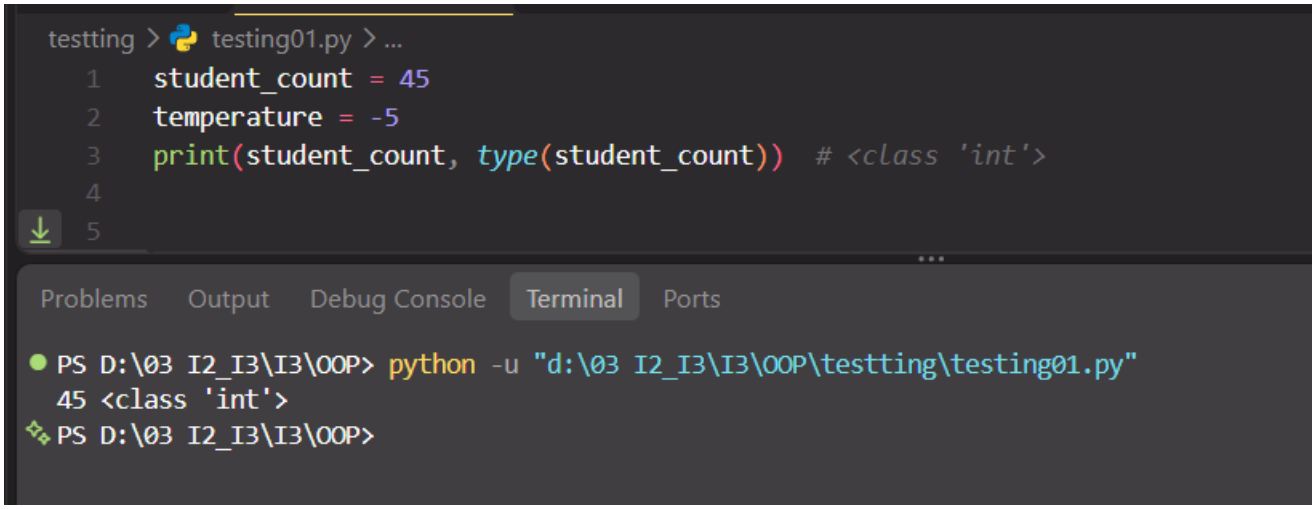
Summary

1. Group 1
    a. Primitive / Built-in Data Types
    b. Collection and Reference Types
    c. Literals in Python
2. Group 2
    a. Local Variables
    b. Instance Attributes
    c. Class Attributes (shared by all instances)
3. Group 3
    a. Public, Non-Public, and Private Naming Conventions
    b. Access Control by Convention and Name Mangling
    c. The @staticmethod Decorator
    d. The @classmethod Decorator
4. Group 4
    e. Abstract Classes and Methods (abc, @abstractmethod)
    f. Access Control and Encapsulation
    g. Other OOP Features: @property, @final, and typing.final
5. Group 5
    a. Arithmetic Operators (+, -, *, /, //, %, **)
    b. Comparison Operators (==, !=, >, <, >=, <=)
    c. Bitwise Operators (&, |, ^, ~, <<, >>)
    d. Logical Operators (and, or, not)
6. Group 6
    h. Assignment Operators (=, +=, -=, *=, etc.)
    i. Special Operators: Identity (is, is not), Membership (in, not in), and Conditional Expressions (x if condition else y)
    j. Operator Precedence and Associativity
7. Group 7
    a. he if Statement
    b. The if-else Statement
    c. The if-elif-else Statement
8. Group 8
    d. Nested if Statements
    e. Multiple-Branch Selection Using match-case (Python 3.10+) or if-elif-else
    f. Conditional Expressions (condition ? value1 : value2 in Java becomes value1 if condition else value2 in Python)
9. Group 9
    a. The while Loop
    b. The for Loop
    c. Repetition with for and while (Python has no native do-while loop)
    d. Loop Control Statements: break, continue, and pass
    e. The break Statement

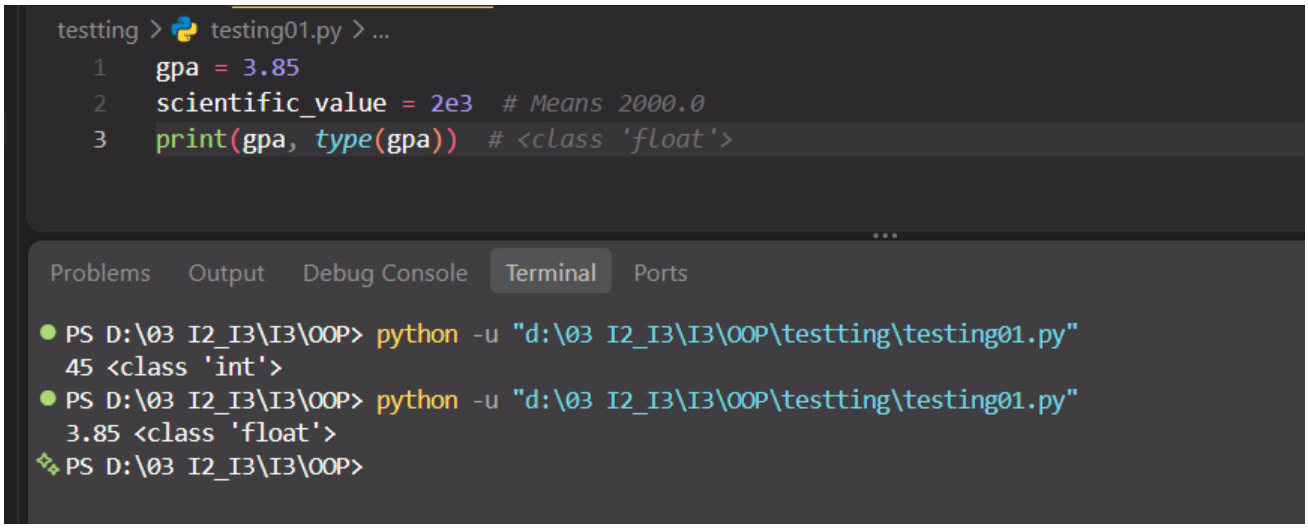
1. Primitive / Built-In Data Types

Primitive types are the atomic building blocks of Python. They store a single, individual value directly in memory.

- ★ **int (Integer):** Used for whole numbers without any decimal point (e.g., 42, -7, 0). They can be arbitrarily large in Python 3.



- ★ **float (Floating-point):** Used for numbers containing a decimal point or written in scientific notation (e.g., 3.14, -0.001, 2e3). They provide fractional precision.



- ★ **bool (Boolean):** Represents truth values: either True or False. Booleans are the core drivers of conditional checking (if-else logic).

```
is_passed = True
is_failed = False
print(is_passed, type(is_passed))  # <class 'bool'>
```

★ **str (String):** Represents a sequence of Unicode characters used for text. Enclosed in single ('...'), double ("..."), or triple quotes.

```python
full_name = "Chhean Ravy"
message = 'Hello, Python!'
print(full_name, type(full_name))  # <class 'str'>
```

Collection and Reference Types

Collection types allow you to group multiple data items into a single container variable.

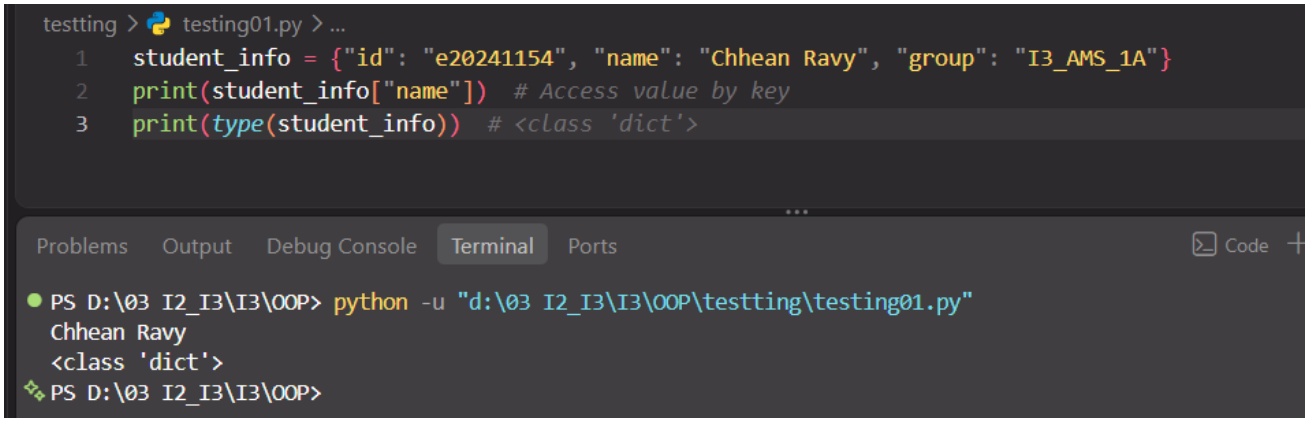
- **list:** An **ordered** and **mutable** (changeable) sequence enclosed in square brackets []. It allows duplicate items and mixed data types.

```python
subjects = ["Math", "Physics", "Python"]
subjects.append("Statistics")  # Mutable: can add items
print(subjects, type(subjects))  # <class 'list'>
```

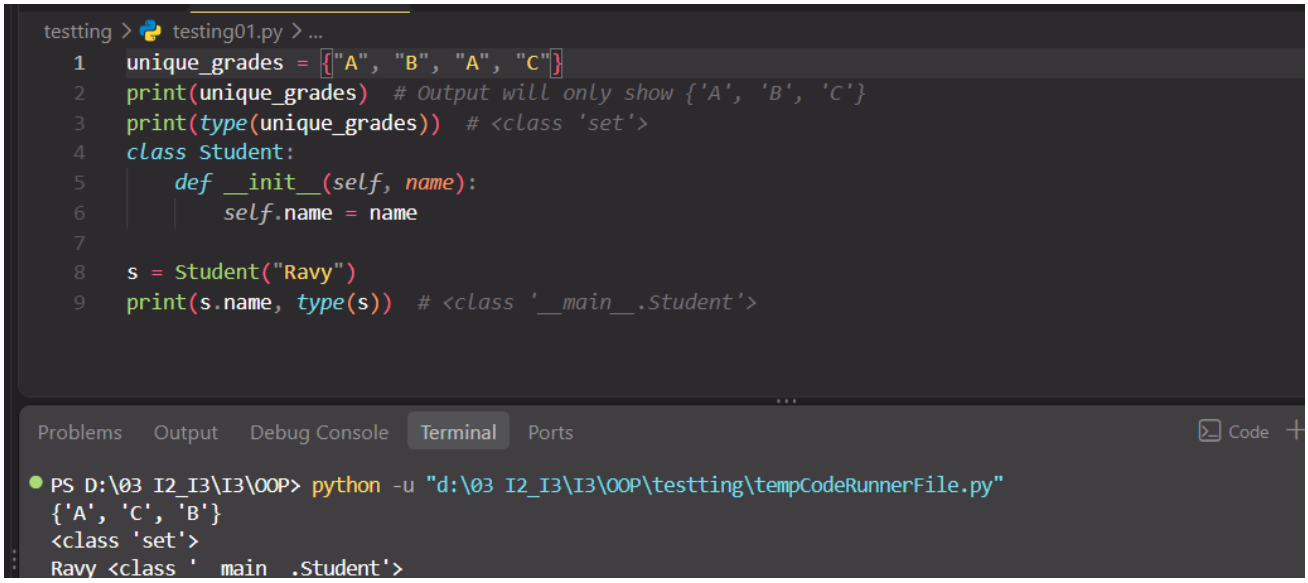
- **tuple:** An **ordered** and **immutable** (unchangeable) sequence enclosed in parentheses (). Once created, its elements cannot be modified.

```python
student_coords = (11.5564, 104.9282)  # Latitude, Longitude
# student_coords[0] = 12.0  # Error: Tuples cannot be modified!
print(student_coords, type(student_coords))  # <class 'tuple'>
```

- **dict (Dictionary):** A mutable collection of **key-value pairs** enclosed in curly braces {}. Keys must be unique and immutable, allowing fast lookups.



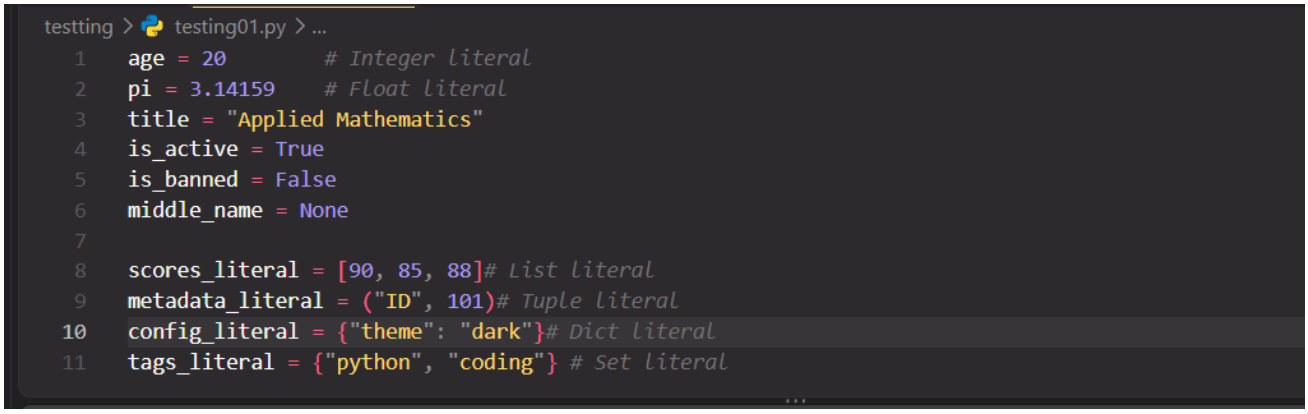
- **set:** An **unordered** and mutable collection enclosed in curly braces {} that strictly contains **unique elements** (duplicates are automatically removed).
- **objects:** Instances of classes that bundle custom data (attributes) and behaviors (methods) together.



Literals in Python

A **literal** is a raw, hard-coded data value assigned directly to a variable or used inline within your source code.

- **Numeric Literals:** Numbers written directly (e.g., 10, 3.14).
- **String Literals:** Text wrapped in quotes (e.g., "Python").
- **Boolean Literals:** The literal keywords True or False.
- **None Literal:** The keyword None, representing a null value or the intentional absence of data.
- **Collection Literals:** Syntactic structures used to initialize collections (e.g., [1, 2, 3] for lists, {"a": 1} for dictionaries).



Comprehensive Practical Example

testting > testing01.py > ...

```python
# Literals & Primitives
name = "Chhean Ravy"     # str literal
student_id = "e20241154"# str literal
age = 21                 # int literal
gpa = 3.9                # float literal
is_active = True         # bool literal
final_score = None       # None literal

# Collections & References
coordinates = (11.5, 104.9)         # Tuple (immutable)
scores = [85, 90, 95, 92]           # List (mutable)
unique_skills = {"Python", "C++", "Python"} # Set (eliminates duplicate "Python")

# Dictionary bundling everything into a student record
student_record = {
    "name": name,
    "id": student_id,
    "age": age,
    "gpa": gpa,
    "active": is_active,
    "location": coordinates,
    "scores": scores,
    "skills": unique_skills,
    "grade": final_score
}

# Output display
print("--- Final Student Record Example ---")
for key, value in student_record.items():
    print(f"{key}: {value} (DataType: {type(value).__name__})")
```

Problems Output Debug Console Terminal Ports

```
PS D:\03 I2_I3\I3\OOP> python -u "d:\03 I2_I3\I3\OOP\testting\testing01.py"
gpa: 3.9 (DataType: float)
active: True (DataType: bool)
location: (11.5, 104.9) (DataType: tuple)
scores: [85, 90, 95, 92] (DataType: list)
skills: {'C++', 'Python'} (DataType: set)
grade: None (DataType: NoneType)
PS D:\03 I2_I3\I3\OOP>
```

Group 2

Local Variables

- **What it is:** A local variable is defined inside a function or a method. It can only be accessed within that specific block of code where it is created.
- **Scope & Lifetime:** Its lifetime begins when the function starts executing and ends when the function exits. It is completely invisible outside of its function.
- **Code Example:**

testting > testing01.py > ...

```python
def calculate_average(score1, score2):
    total = score1 + score2  # 'total' is a local variable
    average = total / 2      # 'average' is a local variable
    return average

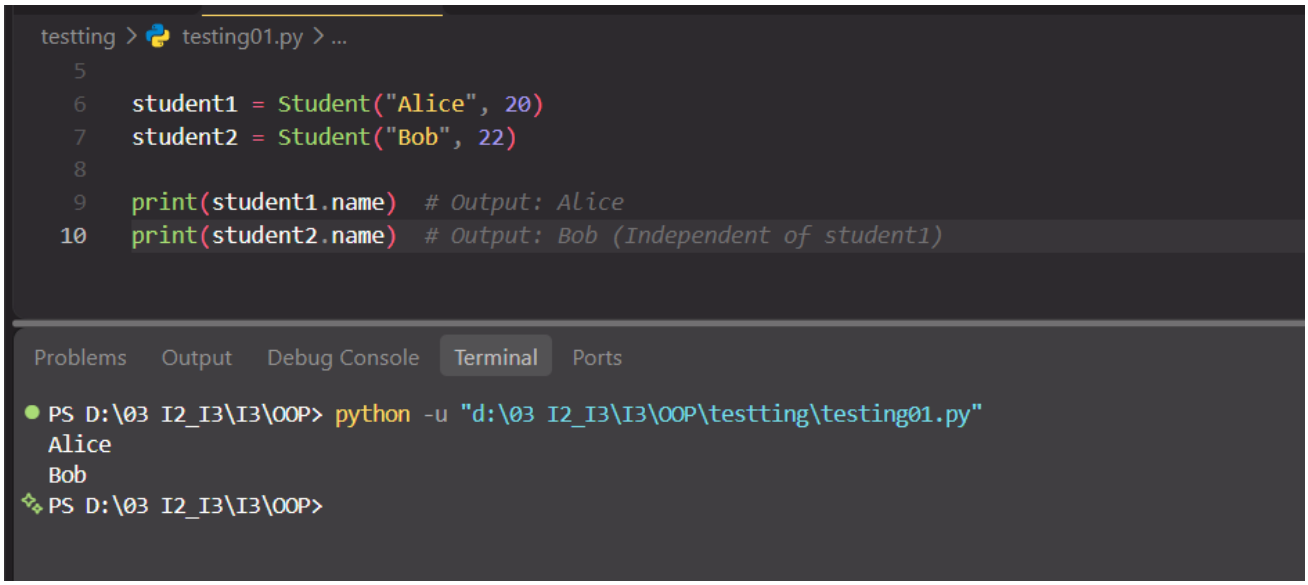
print(calculate_average(85, 95))
# print(total)  # Error! 'total' is not defined outside the function.
```

Problems Output Debug Console Terminal Ports

```
PS D:\03 I2_I3\I3\OOP> python -u "d:\03 I2_I3\I3\OOP\testting\testing01.py"
90.0
PS D:\03 I2_I3\I3\OOP>
```

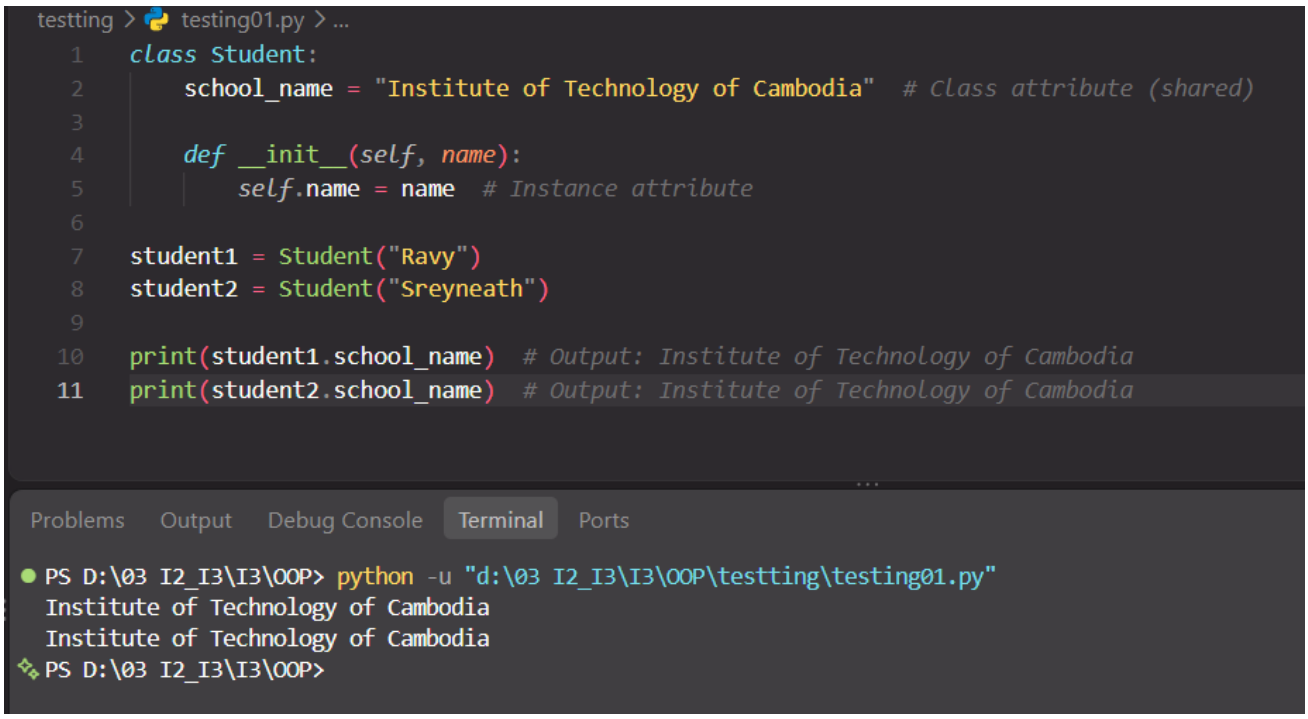
Instance Attributes

- ⇨ **What it is:** Variables that belong to a specific object (instance) created from a class. Every unique object gets its own copy of these attributes.
- ⇨ **How it's defined:** Usually initialized inside the constructor method (`__init__`) using the `self` keyword.
- ⇨ **Code Example:**



Class Attributes

- **What it is:** Variables that are shared across *all* instances (objects) of a class. If you change a class attribute, it reflects across every object referencing it.
- **How it's defined:** Declared directly inside the class body, outside of any methods.
- **Code Example:**



Practical Example: Student Class with Individual Names and a Shared School Name

testting > testing01.py > ...

```python
class Student:
    # Class Attribute (Shared by all students)
    university = "Institute of Technology of Cambodia"
    department = "Applied Mathematics and Statistics"

    def __init__(self, student_id, name, score):
        # Instance Attributes (Unique to each individual student object)
        self.student_id = student_id
        self.name = name
        self.score = score

    def display_profile(self):
        # Local variable inside a method
        status = "Passed" if self.score >= 50 else "Failed"

        print(f"--- Student Profile ---")
        print(f"ID: {self.student_id}")
        print(f"Name: {self.name}")
        print(f"Score: {self.score} ({status})")
        print(f"Department: {Student.department}")
        print(f"University: {Student.university}\n")

# Creating student instances (Objects)
student1 = Student("e20241154", "Chhean Ravy", 92.5)
student2 = Student("e20241155", "Sreyneath Kol", 88.0)

# Displaying profiles showing instance vs class attribute interactions
student1.display_profile()
student2.display_profile()

# Demonstrating that class attributes are shared globally
print(f"Accessing via Class Name: {Student.university}")
```

Group: 3 Python Modifiers and Access Control — Part 1

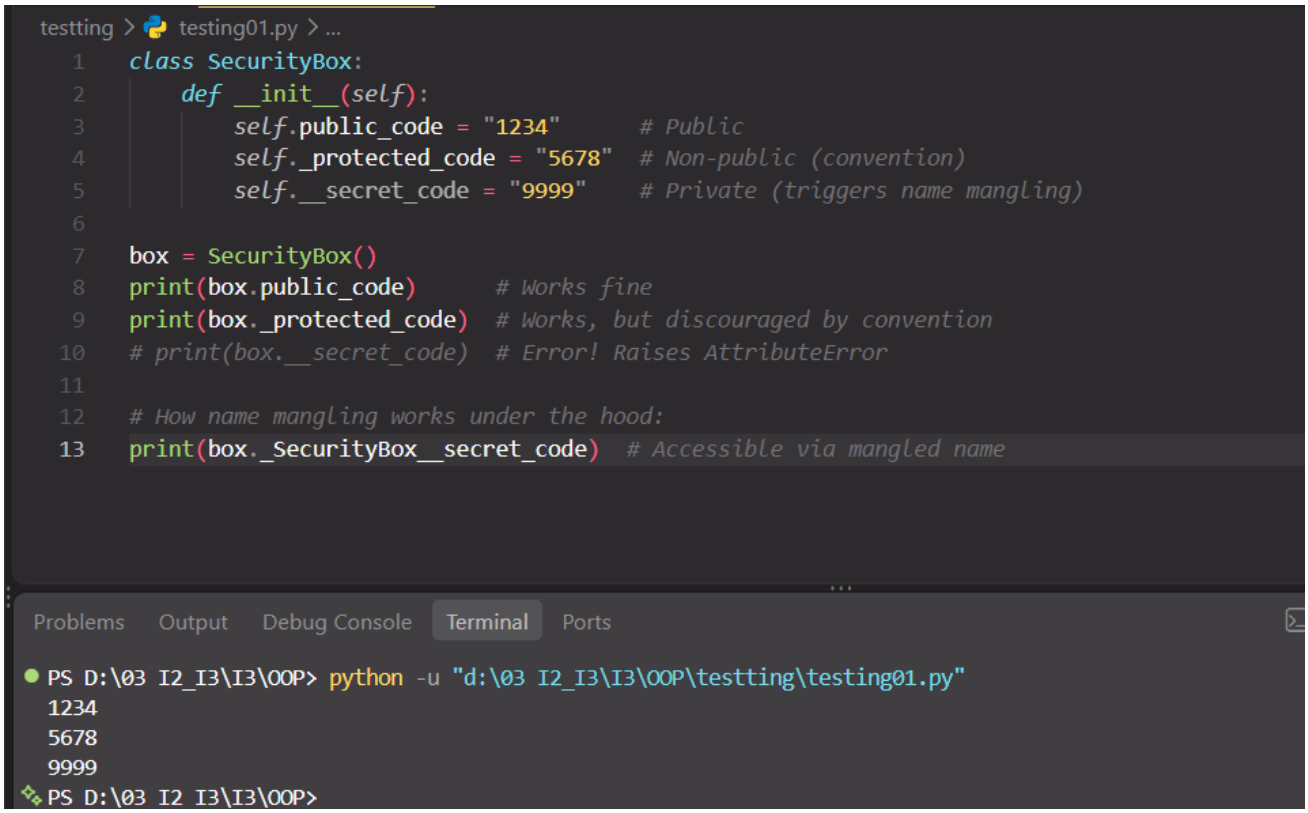
Public, Non-Public, and Private Naming Conventions

Python does not have strict keyword-based modifiers like private, protected, or public found in languages like Java or C++. Instead, it relies entirely on **naming conventions** to signal the intended visibility and usage of variables and methods:

- **Public (name):** Accessible from anywhere, both inside and outside the class. No leading underscores.
- **Non-Public / Protected (_name):** Uses a **single leading underscore**. This is a convention signaling to other programmers that the attribute or method is intended for internal use only (within the class or its subclasses), though Python technically still allows external access.
- **Private (__name):** Uses **double leading underscores**. This triggers Python's **name mangling** mechanism to restrict direct external access.

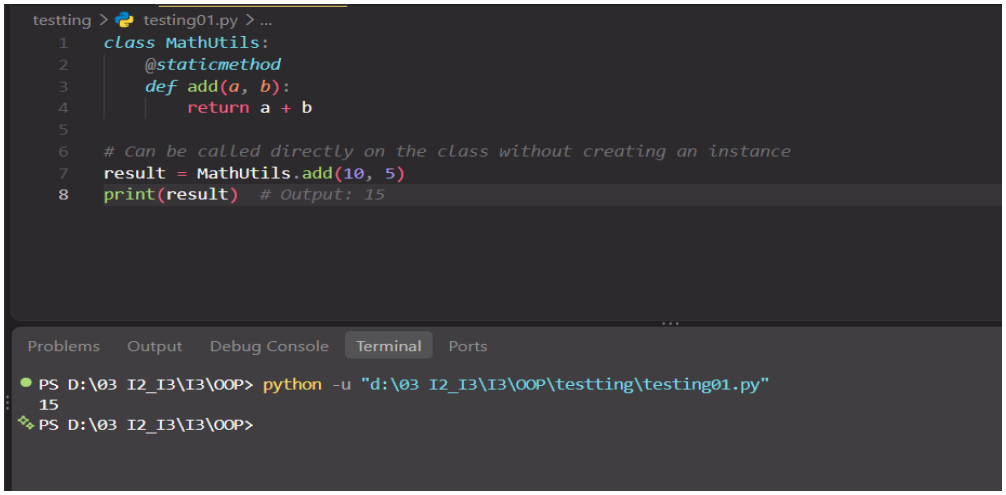
Access Control by Convention and Name Mangling

- **Access Control by Convention (_):** By agreement among Python developers, a single underscore (_variable) means *"Do not touch this from outside unless you know what you are doing."*
- **Name Mangling (__):** When you prefix an attribute or method name with two underscores (__variable), Python automatically alters (mangles) its name inside the class bytecode to _ClassName__variable. This prevents accidental modification or overriding by subclasses, simulating private access.
- **Code Example:**



The @staticmethod Decorator

- **What it is:** A method that belongs to a class rather than any particular object instance. It does not receive self (the object instance) or cls (the class itself) as an implicit first argument.
- **When to use it:** When the method performs a task isolated from class or instance state essentially a utility function that logically lives inside the class for organizational clarity.
- Code Example:



The @classmethod Decorator

- **What it is:** A method that is bound to the **class** rather than its object instance. It takes cls as its first parameter instead of self.
- **When to use it:** When you need a method that can modify or access class-level state, or when implementing **factory methods** that create class instances using alternative data formats.
- **Code Example:**

```
class Student:
    school_name = "ITC"

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name  # Modifies the class attribute

Student.change_school("RUPP")
print(Student.school_name)  # Output: RUPP
```

Practical Example: Controlling Access to a BankAccount Class and Defining Class-Level Methods

```python
class BankAccount:
    # Class Attribute (Shared across all accounts)
    bank_name = "National Savings Bank"
    total_accounts = 0

    def __init__(self, owner, initial_balance):
        self.owner = owner                  # Public attribute
        self._account_type = "Standard"     # Non-public attribute (convention)
        self.__balance = initial_balance    # Private attribute (Name Mangled)

        # Increment class-level tracker
        BankAccount.total_accounts += 1

    # --- Getter method for private balance ---
    def get_balance(self):
        return self.__balance

    # --- Setter method with validation for secure updates ---
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Successfully deposited ${amount}. New Balance: ${self.__balance}")
        else:
            print("Error: Deposit amount must be positive!")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Successfully withdrew ${amount}. Remaining Balance: ${self.__balance}")
        else:
            print("Error: Invalid withdrawal amount or insufficient funds!")

    # --- Class Method: Operates on class attributes ---
    @classmethod
    def get_total_bank_stats(cls):
        return f"Bank: {cls.bank_name} | Total Active Accounts: {cls.total_accounts}"

    # --- Static Method: Utility function independent of instance/class state ---
    @staticmethod
    def validate_interest_rate(rate):
        return 0.0 <= rate <= 0.05


# --- Execution and Demonstration ---
print("--- Initializing Banking System ---")
account1 = BankAccount("Chhean Ravy", 500.0)
account2 = BankAccount("Sreyneath Kol", 1200.0)

# 1. Accessing public and convention-based attributes
print(f"Owner: {account1.owner}")
print(f"Account Type (Protected Convention): {account1._account_type}")

# 2. Accessing private balance via secure getter & setter methods
print(f"Initial Balance: ${account1.get_balance()}")
account1.deposit(150.0)
account1.withdraw(200.0)

# Demonstrating name mangling enforcement (direct access fails)
try:
    print(account1.__balance)
except AttributeError:
    print("Security Check: Direct access to __balance blocked by Name Mangling!")

# 3. Using Class Method
print("\n--- Class Method Output ---")
print(BankAccount.get_total_bank_stats())

# 4. Using Static Method
print("\n--- Static Method Output ---")
is_valid = BankAccount.validate_interest_rate(0.03)
print(f"Is interest rate 3% valid? {is_valid}")
```

Abstract Classes and Methods (abc, @abstractmethod)

- ★ **What it is:** An abstract class serves as a blueprint for other classes. It cannot be instantiated directly. Instead, its purpose is to define common methods that child classes *must* implement.
- ★ **How it works:** Python uses the built-in abc (Abstract Base Classes) module. By using the @abstractmethod decorator, you force any subclass to provide its own concrete implementation of that method; otherwise, Python raises an error.
- ★ **Code Example:**

```python
from abc import ABC, abstractmethod

class Animal(ABC): # Abstract Class
    @abstractmethod
    def make_sound(slef):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

# a = Animal()  # Error! Cannot instantiate abstract class.
d = Dog()       # Valid because Dog implements make_sound()
print(d.make_sound())
```

f. Access Control and Encapsulation

- **What it is:** Encapsulation is the practice of hiding the internal state and requiring all interaction to be performed through controlled methods (getters and setters).
- **Why it matters:** It prevents unauthorized direct modification of sensitive data, ensuring object integrity. In Python, this is achieved combining private/protected attributes (__, _) with property decorators.

g. Other OOP Features: @property, @final, and typing.final

- **@property:** Allows you to define a method that can be accessed like an attribute (without parentheses). It is commonly used to implement safe getters and setters cleanly.
- **@final / typing.final:** Introduced in Python 3.8 (typing.final), this decorator prevents a method from being overridden by subclasses, or a class from being subclassed. It enforces architectural design rules.

- **Code Example:**

```python
from typing import final

class Base:
    @final
    def secure_method(self):
        return "Cannot be overridden!"

# class Child(Base):
#     def secure_method(self):  # Error! Triggers type-checking
#         pass
```

Practical Example: Abstract Shape Class with Circle and Rectangle Subclasses

Below is a complete, executable Python script demonstrating abstract base classes, abstract methods, @property encapsulation, and subclass polymorphism:

```python
from abc import ABC, abstractmethod
import math

# 1. Abstract Base Class
class Shape(ABC):
    def __init__(self, color):
        self._color = color  # Protected attribute

    # Abstract methods that subclasses MUST implement
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    # Concrete property accessible like an attribute
    @property
    def color(self):
        return self._color


# 2. Concrete Subclass: Circle
class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)
        self.__radius = radius  # Private attribute

    @property
    def radius(self):
        return self.__radius

    # Implementing abstract area method
    def area(self):
        return math.pi * (self.__radius ** 2)

    # Implementing abstract perimeter method
    def perimeter(self):
        return 2 * math.pi * self.__radius


# 3. Concrete Subclass: Rectangle
class Rectangle(Shape):
    def __init__(self, color, width, height):
        super().__init__(color)
        self.__width = width
        self.__height = height

    # Implementing abstract area method
    def area(self):
        return self.__width * self.__height

    # Implementing abstract perimeter method
    def perimeter(self):
        return 2 * (self.__width + self.__height)


# --- Execution and Demonstration ---
print("--- Abstract Shape Polymorphism Example ---")

# Instantiating subclasses
my_circle = Circle("Red", 5.0)
my_rect = Rectangle("Blue", 4.0, 6.0)

# Displaying Circle Details
print(f"Shape: Circle | Color: {my_circle.color}")
print(f"Radius: {my_circle.radius}")
print(f"Area: {my_circle.area():.2f}")
print(f"Perimeter: {my_circle.perimeter():.2f}\n")

# Displaying Rectangle Details
print(f"Shape: Rectangle | Color: {my_rect.color}")
print(f"Area: {my_rect.area():.2f}")
print(f"Perimeter: {my_rect.perimeter():.2f}")
```

Group: 5 Python Operators — Part 1

a. Arithmetic Operators (+, -, *, /, //, %, **)

Arithmetic operators are used with numeric values to perform common mathematical calculations.

- **Addition (+):** Adds values together (e.g., 10 + 5 $\rightarrow$ 15).
- **Subtraction (-):** Subtracts the right operand from the left (e.g., 10 - 5 $\rightarrow$ 5).
- **Multiplication (*):** Multiplies values (e.g., 10 * 5 $\rightarrow$ 50).
- **Division (/):** Divides left operand by the right, always returning a float (e.g., 10 / 4 $\rightarrow$ 2.5).
- **Floor Division (//):** Divides and rounds down to the nearest whole integer (e.g., 10 // 4 $\rightarrow$ 2).
- **Modulo (%):** Returns the remainder of division (e.g., 10 % 3 $\rightarrow$ 1).
- **Exponentiation (**):** Raises the left operand to the power of the right (e.g., 2 ** 3 $\rightarrow$ 8).

b. Comparison Operators (==, !=, >, <, >=, <=)

Comparison operators are used to compare two values. They always evaluate to a Boolean (True or False).

- **Equal (==):** Returns True if operands are equal (5 == 5).
- **Not Equal (!=):** Returns True if operands are not equal (5 != 3).
- **Greater Than (>):** 10 > 5 $\rightarrow$ True.
- **Less Than (<):** 3 < 5 $\rightarrow$ True.
- **Greater Than or Equal To (>=):** 5 >= 5 $\rightarrow$ True.
- **Less Than or Equal To (<=):** 4 <= 5 $\rightarrow$ True.

c. Bitwise Operators (&, |, ^, ~, <<, >>)

Bitwise operators act on operands as if they were strings of binary digits (bits). They operate bit by bit.

- **AND (&):** Sets each bit to 1 if both bits are 1.
- **OR (|):** Sets each bit to 1 if one of two bits is 1.
- **XOR (^):** Sets each bit to 1 if only one of two bits is 1.
- **NOT (~):** Inverts all the bits.
- **Zero fill left shift (<<):** Shift left by pushing zeros in from the right.
- **Signed right shift (>>):** Shift right by pushing copies of the leftmost bit in from the left.

d. Logical Operators (and, or, not)

Logical operators are used to combine conditional statements.

- **and:** Returns True if **both** statements are true (x > 5 and y < 10).
- **or:** Returns True if **at least one** statement is true (x > 5 or y < 2).
- **not:** Reverses the result, returns False if the result is true (not(x > 5)).

Practical Example: Calculating Student Grades and Checking Eligibility

Below is a complete, executable Python script demonstrating arithmetic, comparison, and logical operators in a realistic academic scenario:

testting > testing01.py > ...

```python
# --- 1. Arithmetic Operators: Calculating Total & Average Scores ---
quiz_score = 18
midterm_score = 42
final_exam_score = 75

total_score = quiz_score + midterm_score + final_exam_score  # Addition
max_possible_score = 150

# Calculate percentage using division and multiplication
percentage = (total_score / max_possible_score) * 100

# Floor division and Modulo example
points_bucket = total_score // 10  # Grouping score into tens
score_remainder = total_score % 10

print("--- Academic Score Summary ---")
print(f"Total Score: {total_score} / {max_possible_score}")
print(f"Percentage: {percentage:.2f}%")
print(f"Score Bucket (Tens): {points_bucket}, Remainder: {score_remainder}\n")


# --- 2. Comparison & Logical Operators: Checking Pass & Scholarship Eligibility ---
attendance_rate = 0.92  # 92% attendance
minimum_passing_score = 50.0
minimum_attendance = 0.80

# Comparison operations returning Booleans
has_passed_exam = percentage >= minimum_passing_score
has_good_attendance = attendance_rate >= minimum_attendance

# Logical 'and' operator: Student qualifies for scholarship only if BOTH conditions are met
qualifies_for_scholarship = (percentage >= 85.0) and has_good_attendance

# Logical 'or' operator: Needs academic review if score is low OR attendance is poor
needs_academic_review = (percentage < minimum_passing_score) or (attendance_rate < 0.70)

# Logical 'not' operator
is_failing = not has_passed_exam

print("--- Eligibility Evaluation ---")
print(f"Passed Exam: {has_passed_exam}")
print(f"Good Attendance: {has_good_attendance}")
print(f"Qualifies for Scholarship: {qualifies_for_scholarship}")
print(f"Needs Academic Review: {needs_academic_review}")
print(f"Is Failing: {is_failing}")
```

Group: 6 Python Operators — Part 2

e. Assignment Operators (=, +=, -=, *=, etc.)

Assignment operators are used to assign values to variables and perform in-place arithmetic updates simultaneously.

- **Simple Assignment (=):** Assigns a right-side value to a left-side variable (x = 10).
- **Add and Assign (+=):** Adds the right value to the variable and assigns the result back (x += 5 is equivalent to x = x + 5).
- **Subtract and Assign (-=)**: Subtracts and assigns (x -= 2).
- **Multiply and Assign (*=):** Multiplies and assigns (x *= 3).
- **Divide and Assign (/=):** Divides and assigns (x /= 2).
- **Floor Divide, Modulo, and Exponent Assign (//=, %=, **=):** Perform respective operations in-place.

f. Special Operators: Identity, Membership, and Conditional Expressions

- **Identity Operators (is, is not):**

- Compare the **memory locations** (identity) of two objects, rather than their values.
- x is y returns True if both variables point to the exact same object in memory.

- **Membership Operators (in, not in):**
  - Test for membership within a sequence or collection (such as strings, lists, tuples, or dictionaries).
  - "Apple" in ["Apple", "Banana"] returns True.
- **Conditional Expressions (Ternary Operator):**
  - A compact way to write conditional statements in a single line: x if condition else y.
  - Evaluates to x if the condition is True, otherwise evaluates to y.

g. Operator Precedence and Associativity

- **Precedence:** Determines the order in which different operators are evaluated in an expression. For example, multiplication (*) and division (/) have higher precedence than addition (+) and subtraction (-), so they are executed first. (Remember the standard mathematical order: Parentheses $\rightarrow$ Exponents $\rightarrow$ Multiplication/Division $\rightarrow$ Addition/Subtraction).
- **Associativity:** Determines the order of evaluation for operators with the **same precedence level**. Most operators in Python are **left-to-right associative** (evaluated from left to right), except for exponentiation (**), which is **right-to-left associative**.

Practical Example: Updating Object Attributes, Checking Membership, and Evaluating Expressions

Below is a complete, executable Python script demonstrating assignment operators, identity checks, membership tests, conditional expressions, and operator precedence:

testting > testing01.py > ...

```python
# --- 1. Assignment Operators & In-Place Updates ---
student_score = 80
student_score += 10  # Equivalent to student_score = student_score + 10 (Now 90)
student_score *= 1.05  # Applying 5% bonus credit (Now 94.5)

print("--- Assignment Updates ---")
print(f"Updated Score: {student_score}\n")


# --- 2. Identity and Membership Operators ---
valid_courses = ["Python 101", "Data Structures", "Applied Mathematics"]
enrolled_course = "Python 101"

# Membership check
is_enrolled = enrolled_course in valid_courses

# Identity check (comparing object references)
default_score = student_score
is_same_reference = (student_score is default_score)

print("--- Special Operators Check ---")
print(f"Is '{enrolled_course}' a valid course? {is_enrolled}")
print(f"Does student_score share the exact memory reference? {is_same_reference}\n")


# --- 3. Conditional Expressions (Ternary Operator) ---
# Syntax: value_if_true if condition else value_if_false
performance_status = "Excellent" if student_score >= 90 else "Satisfactory"

print("--- Conditional Expression ---")
print(f"Performance Status: {performance_status}\n")


# --- 4. Operator Precedence and Evaluation ---
# Multiplication and division take precedence over addition and subtraction
# Expression: 10 + 5 * 2 ** 3 / 2
# Step-by-step:
# 1. Exponent: 2 ** 3 = 8
# 2. Multiplication: 5 * 8 = 40
# 3. Division: 40 / 2 = 20
# 4. Addition: 10 + 20 = 30
complex_calculation = 10 + 5 * 2 ** 3 / 2

print("--- Precedence Evaluation ---")
print(f"Result of 10 + 5 * 2 ** 3 / 2 = {complex_calculation}")
```

Group: 7 Python Decision-Making Statement

a. The if Statement

- **What it is:** The simplest decision-making statement. It evaluates a Boolean condition. If the condition evaluates to True, the indented block of code underneath it runs. If False, the code is skipped entirely.
- **Syntax & Concept:**

```python
if condition:
    # Code to execute if condition is True
```

b. The if-else Statement

- **What it is:** Used when you want to execute one block of code if a condition is true, and a completely different block of code if that condition is false.
- **Syntax & Concept:**

```
if condition:
    # Code if condition is True
else:
    # Code if condition is False
```

c. The if-elif-else Statement

- **What it is:** Used when you need to test **multiple conditions sequentially**. elif stands for "else if". Python checks each condition from top to bottom; the moment one evaluates to True, that block executes, and the rest of the structure is skipped. The final else acts as a catch-all if none of the preceding conditions match.
- **Syntax & Concept:**

```
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition2 is True
else:
    # Code if none of the conditions are True
```

Practical Example: Determining Student Pass/Fail Status and Assigning Letter Grades

testting > testing01.py > ...

```python
# --- 1. Basic 'if' Statement: Checking Minimum Submission ---
assignment_submitted = True

print("--- Submission Check ---")
if assignment_submitted:
    print("Assignment received successfully.")


# --- 2. 'if-else' Statement: Determining Pass/Fail Status ---
student_score = 68.5
passing_threshold = 50.0

print("\n--- Pass/Fail Evaluation ---")
if student_score >= passing_threshold:
    print(f"Status: PASS (Score: {student_score})")
else:
    print(f"Status: FAIL (Score: {student_score})")


# --- 3. 'if-elif-else' Statement: Assigning Letter Grades ---
# Grading Scale:
# >= 90: A
# >= 80: B
# >= 70: C
# >= 50: D
# < 50:  F

print("\n--- Letter Grade Assignment ---")
if student_score >= 90.0:
    letter_grade = "A"
elif student_score >= 80.0:
    letter_grade = "B"
elif student_score >= 70.0:
    letter_grade = "C"
elif student_score >= 50.0:
    letter_grade = "D"
else:
    letter_grade = "F"

print(f"Final Score: {student_score} | Assigned Letter Grade: {letter_grade}")
```

Group: 8 Python Decision-Making — Part 2

d. Nested if Statements

- **What it is:** Placing an if statement (or if-else statement) inside another if or else block.
- **When to use it:** Used when you need to check a secondary condition *only after* a primary condition has already evaluated to True.
- **Code Example:**

```python
score = 88
is_honor_student = True

if score >= 80:
    print("Passed the course.")
    if is_honor_student:
        print("Eligible for Dean's List honors!")
```

e. Multiple-Branch Selection Using `match-case` (Python 3.10+)

- **What it is:** Introduced in Python 3.10, the structural pattern matching match-case statement provides an elegant and readable way to handle multiple branches, similar to switch-case statements in languages like C++ or Java.
- **When to use it:** When matching a variable against multiple exact values or structural patterns, keeping your code cleaner than long chains of elif.
- **Code Example:**

```
grade_rank = "A"

match grade_rank:
    case "A":
        print("Outstanding performance!")
    case "B":
        print("Good job!")
    case "C" | "D":
        print("Needs improvement.")
    case _:
        print("Invalid grade rank.")
```

f. Conditional Expressions (Ternary Operator)

- **What it is:** A compact, one-line expression for evaluating conditions. In languages like C/Java/JavaScript, you use the ternary syntax condition ? value1 : value2. In Python, this is written as **value1 if condition else value2**.
- **Code Example:**

```
score = 75
status = "Pass" if score >= 50 else "Fail"
print(status)  # Output: Pass
```

Practical Example: A Menu-Driven Student Management System

Below is a complete, executable Python script combining nested if statements, match-case branch selection, and conditional expressions in a realistic interactive menu system:

Welcome testing01.py U

testing > testing01.py > ...

```python
# --- Interactive Menu-Driven Student Management System ---

# Simulated student database dictionary
student_record = {
    "id": "e20241154",
    "name": "Chhean Ravy",
    "score": 88.5,
    "status": "Active"
}

print("=== Student Management System Menu ===")
print("1. View Student Profile")
print("2. Update Student Score")
print("3. Check Scholarship Eligibility")
print("4. Exit")

# Simulating user menu choice (Option 3 selected for demonstration)
user_choice = 3

match user_choice:
    case 1:
        print("\n--- Student Profile ---")
        print(f"ID: {student_record['id']}")
        print(f"Name: {student_record['name']}")
        print(f"Score: {student_record['score']}")

    case 2:
        print("\n--- Update Score ---")
        new_score = 92.0
        student_record["score"] = new_score
        print(f"Score updated successfully to {new_score}.")

    case 3:
        print("\n--- Scholarship Eligibility Check ---")
        current_score = student_record["score"]
        is_active_student = (student_record["status"] == "Active")

        # Nested if & Conditional expression combined
        if is_active_student:
            scholarship_tier = "Gold Tier" if current_score >= 90 else ("Silver Tier" if current_score >= 80 else "None")
            print(f"Student: {student_record['name']}")
            print(f"Assigned Scholarship Status: {scholarship_tier}")
        else:
            print("Error: Student is inactive. Not eligible for scholarship review.")

    case 4:
        print("\nExiting system. Goodbye!")

    case _:
        print("\nError: Invalid menu selection. Please choose between 1 and 4.")
```

Group 9: Python Loops and Loop Control

a. The `while` Loop

- **What it is:** Executes a block of code repeatedly as long as a given Boolean condition evaluates to `True`.
- **When to use it:** Ideal when the number of iterations is unknown beforehand and depends on a dynamic condition changing inside the loop.
- **Code Example:**

```python
count = 3
while count > 0:
    print(f"Countdown: {count}")
    count -= 1
```

b. The `for` Loop

- **What it is:** Used for iterating over a sequence (such as a list, tuple, dictionary, set, or string).
- **When to use it:** Ideal when you want to loop through a collection of items or execute a block of code a specific, known number of times (often combined with range()).
- **Code Example:**

```
for subject in ["Math", "Physics", "Python"]:
    print(f"Studying: {subject}")
```

c. Repetition with for and while (No Native do-while Loop)

- **Concept:** Unlike languages like C++ or Java, which feature a `do-while` loop (guaranteeing code execution at least once before checking the condition), **Python does not have a native `do-while` loop**.
- **Workaround:** To replicate a `do-while` behavior in Python, developers use a `while True` loop combined with a conditional `break` statement inside.

d. Loop Control Statements: `break`, `continue`, and `pass`

- **`break`:** Immediately terminates the entire loop, skipping any remaining iterations.
- **`continue`:** Skips the current iteration's remaining code and jumps straight to the next iteration of the loop.
- **`pass`:** A null statement that acts as a placeholder. It does nothing syntactically, preventing errors when a block of code is required by Python syntax but no logic is written yet.

e. The `break` Statement (In-Depth)

- **Concept:** When triggered inside a `for` or `while` loop, execution breaks out of the enclosing loop immediately. Often used in search operations when a target item is found to prevent unnecessary looping.

Practical Example: Iterating Through Student Objects, Calculating Averages, and Searching for a Student

Welcome testing01.py U

testting > testing01.py > ...

```python
# --- 1. Dataset of Student Records ---
students_db = [
    {"id": "e20241154", "name": "Chhean Ravy", "score": 92.5},
    {"id": "e20241155", "name": "Sreyneath Kol", "score": 88.0},
    {"id": "e20241156", "name": "Heng Piseth", "score": 45.0},  # Failing score
    {"id": "e20241157", "name": "Vatey Chan", "score": 95.5}
]

# --- 2. Using a 'for' Loop & 'continue' to Calculate Average (Skipping Failures) ---
total_score = 0
valid_count = 0

print("--- Calculating Average Score (Passing Students Only) ---")
for student in students_db:
    if student["score"] < 50:
        print(f"Skipping {student['name']} (Score below passing threshold).")
        continue  # Skip current iteration if student failed

    total_score += student["score"]
    valid_count += 1

average_score = total_score / valid_count if valid_count > 0 else 0
print(f"Class Average (Passing): {average_score:.2f}\n")


# --- 3. Using a 'for' Loop & 'break' to Search for a Specific Student ---
target_id = "e20241155"
found_student = None

print(f"--- Searching Database for ID: {target_id} ---")
for student in students_db:
    if student["id"] == target_id:
        found_student = student
        break  # Terminate loop immediately once target is found

if found_student:
    print(f"Match Found! Name: {found_student['name']} | Score: {found_student['score']}\n")
else:
    print("Student ID not found in database.\n")


# --- 4. Replicating 'do-while' Behavior in Python using 'while True' ---
print("--- Interactive Retry Loop (Simulating do-while) ---")
attempt_count = 0

while True:
    attempt_count += 1
    print(f"Connection attempt #{attempt_count}...")

    # Simulate successful connection on attempt 2
    if attempt_count >= 2:
        print("Connection established successfully!")
        break  # Exit the infinite loop
```